

[Wiki »](#)

Лабораторная работа №3

Одним из основных ресурсов компьютера является центральный процессор. Для эффективного использования процессора операционная система должна управлять задачами, которые выполняются на нем. Современные компьютеры, как правило, должны выполнять несколько задач одновременно. В случае с персональным компьютером помимо той программы, с которой в данный момент осуществляется взаимодействие, могут выполняться программы обмена сообщениями, музыкальные проигрыватели, программы синхронизации с облачным хранилищем и т. д. Веб-сервер может обрабатывать сразу несколько запросов, часть из которых может ожидать обмена данными с диском. Даже вычислительные программы для более полного использования современных многоядерных процессоров рекомендуется по возможности разделять на несколько одновременно выполняющихся вычислений.

С точки зрения операционной системы для управления одновременно выполняющимися задачами используется модель процессов. При этом процессор переключается между процессами по определенному алгоритму, заданному операционной системой в виде планировщика процессов, и предоставляет каждому процессу определенный интервал времени для выполнения. Процесс — это экземпляр выполняемой программы, включающий в себя помимо ее кода значения регистров процессора и прочие необходимые для выполнения данные. Концептуально каждый процесс использует свой собственный виртуальный процессор.

В этой и следующих работах в JOS добавляется возможность запускать одновременно несколько процессов, а также все сопутствующие возможности, требуемые для их работы (управление процессами, планировщик процессов и т. д.), причём все процессы работают в едином адресном пространстве, то есть могут иметь прямой доступ к памяти друг друга и к памяти ядра ОС. В связи с этим ядро и процессы не могут быть защищены друг от друга, и любая ошибка в программе может привести к порче данных ядра или другого процесса, что не критично для учебной ОС, но недопустимо для настоящей. Данная проблема будет решена в дальнейших работах.

Для получения файлов и изменений, необходимых для выполнения работы, следует обновить удаленный репозиторий, создать новую ветвь под названием `working-lab3`, после чего выполнить слияние с ней ветви `lab3`, которая появилась в репозитории, например:

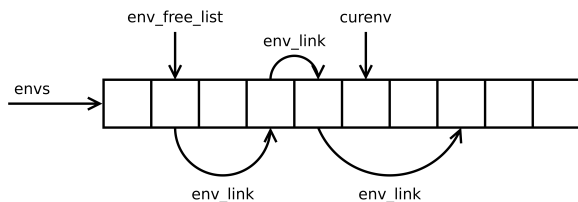
```
$ git pull
$ git checkout lab3
$ git checkout working-lab2
$ git checkout -b working-lab3
$ git merge lab3
```

При возникновении конфликтов (случаев, когда при выполнении предыдущих работ вы изменили файлы, изменившиеся в ветке `lab3`) команда `git merge` сообщит, какие файлы находятся в состоянии конфликта, и перед выполнением работы вам потребуется избавиться от конфликта, отредактировав эти файлы, и выполнить `git commit -a`.

Эта лабораторная работа посвящена реализации основных возможностей ядра, необходимых для запуска процессов в режиме ядра без виртуальной памяти. В частности, нужно реализовать создание в ядре JOS структур для отслеживания процессов, функции создания нового процесса, загрузки образа программы и запуска процесса. Процессы на данном этапе будут функционировать в режиме ядра. Некоторые участки кода защищены макросом `CONFIG_KSPACE`. Это необходимо для того, чтобы в дальнейшем, когда процессы будут работать в пользовательском режиме, этот специфичный код не использовался или использовался его альтернативный вариант.

В этой работе может быть полезной возможность GCC встраивать в код программы фрагменты на языке ассемблера — для этого используется оператор `asm`. Работа может быть выполнена и без их использования, однако такие фрагменты уже присутствуют в исходном коде, и их необходимо понимать.

Новый заголовочный файл `inc/env.h` содержит основные определения для процессов в JOS. Ядро использует структуру `Env` (от `environment` — окружение; название подчеркивает отличие интерфейса управления процессами от традиционного, используемого в UNIX-подобных операционных системах) для отслеживания каждого процесса. В результате выполнения этой работы будет создан только один процесс, но возможность создания нескольких процессов будет добавлена.



Как можно видеть в `kern/env.c`, ядро поддерживает три основных глобальных переменных, относящихся к процессам:

```
struct Env *envs; // Массив всех процессов, используемых и свободных
struct Env *curenv = NULL; // Указатель на текущий процесс
static struct Env *env_free_list; // Список свободных процессов
```

После того, как JOS запущена, указатель `envs` указывает на массив структур `Env`, соответствующих всем возможным процессам в системе. Ядро JOS поддерживает максимум `NENV` (константа, определенная в `inc/env.h`) одновременно активных процессов, хотя обычно таких процессов гораздо меньше. После того, как массив `envs` выделен, он содержит один экземпляр структуры `Env` для каждого из `NENV` возможных процессов.

Ядро JOS содержит все неактивные структуры `Env` в связном списке `env_free_list`, что дает возможность легкого выделения и освобождения процессов путем их добавления в список и удаления из него.

Ядро также использует `curenv` для отслеживания выполняемого в данный момент процесса. Во время загрузки ядра, до первого запуска процесса, `curenv` равен `NULL`.

Структура `Env` определена в `inc/env.h` следующим образом:

```
struct Env {
    struct Trapframe env_tf;
    struct Env *env_link;
    env_id_t env_id;
    env_id_t env_parent_id;
    enum EnvType env_type;
    unsigned env_status;
    uint32_t env_runs;
```

```
uint8_t *binary;
};
```

Рассмотрим подробнее поля этой структуры:

- env_tf:**
Структура, определенная в inc/trap.h. Содержит сохраненные значения регистров для процесса в момент, когда процесс не выполняется, а выполняется код ядра или другой процесс. Ядро сохраняет эти значения при переключении процессов, чтобы работа процесса в дальнейшем могла быть возобновлена в том же месте, где она была прервана.
- env_link:**
Ссылка на следующий Env в env_free_list. Сама переменная env_free_list указывает на первый свободный процесс в списке.
- env_id:**
Значение, однозначно идентифицирующее процесс, который в настоящее время использует данную структуру Env (т.е. данный слот в массиве envs). После завершения процесса ядро может использовать ту же структуру Env для нового процесса, но новый процесс будет иметь другое значение env_id.
- env_parent_id:**
Идентификатор (env_id) родительского процесса, т.е. процесса, который создал данный. Таким образом, процессы формируют «родословную», которая может быть полезна, например, для контроля доступа.
- env_type:**
Тип процесса. В настоящий момент это поле может принимать только одно значение ENV_TYPE_KERNEL. В дальнейшем будет введено еще несколько типов процессов.
- env_status:**
Статус процесса. Принимает одно из следующих значений:
 - ENV_FREE: структура неактивна и, следовательно, находится в env_free_list.
 - ENV_RUNNABLE: структура соответствует процессу, который ждет возможности работать на процессоре.
 - ENV_RUNNING: структура соответствует процессу, работающему в данный момент.
 - ENV_NOT_RUNNABLE: структура соответствует процессу, который активен, но в данный момент не готов к работе: например, ожидает межпроцессного взаимодействия (IPC) с другим процессом.
 - ENV_DYING: структура соответствует зомби-процессу. Зомби-процесс будет освобожден в следующий раз, когда он вызовет исключение ядра. В данной работе это значение не используется.
- binary:**
Указатель на начало исполняемого файла в ядре для файлов, включенных в образ ядра.

Как и Unix-процесс, процесс JOS связывает понятия «поток» и «адресное пространство». Поток определяется прежде всего сохраненными регистрами (поле env_tf), а адресное пространство — закрепленной за каждым процессом индивидуальной областью памяти. Области памяти разных процессов не должны пересекаться. Сами адреса, которые закреплены за процессом, задаются компоновщиком при сборке. Чтобы запустить процесс на выполнение, ядро должно установить сохраненные значения регистров процессора.

Создание и запуск процесса

Теперь следует написать в kern/env.c код, необходимый для запуска процесса. Так как файловая система отсутствует, ядро загружает статический двоичный образ программы в формате ELF, встроенный в само ядро.

GNUmakefile создает ряд двоичных образов в obj/prog/, а в kern/Makefrag эти файлы встраиваются непосредственно в ядро, как если бы они были файлами .o. Параметр командной строки компоновщика -b binary заставляет его вставлять эти файлы в образ ядра в том виде, в котором они находятся на диске, никак их не интерпретируя и не изменяя; с точки зрения компоновщика эти файлы могут являться даже не ELF-образами, а, например, текстовыми файлами или фотографиями. В файле obj/kern/kernel.sym после сборки ядра можно видеть ряд символов, соответствующих этим файлам, с названиями вида _binary_obj_prog_test1_start, _binary_obj_prog_test1_end и _binary_obj_prog_test1_size. Эти символы обеспечивают возможность ссылаться на эти встроенные двоичные файлы в обычном коде ядра.

В функции i386_init() в kern/init.c находится код для запуска этих двоичных образов в качестве процессов. Тем не менее, критически важные функции настройки процессов не завершены, вы должны написать их самостоятельно.

Упражнение №1

В файле env.c допишите следующие функции:

- env_init()**
Инициализирует все структуры Env в массиве envs и добавляет их в env_free_list. Настраивает оборудование для сегментации с отдельными сегментами для уровня привилегий 0 (ядро) и 3 (пользователь).
- load_icode()**
Декодирует двоичный ELF-образ так же, как это уже делает загрузчик, и загружает его содержимое в адресное пространство нового процесса. Пока не следует обращать внимания на функцию bind_functions.
- env_create()**
Выделяет процесс с помощью env_alloc и загружает в него двоичный ELF-образ путем вызова load_icode.
- env_run()**
Запускает процесс.

В этих функциях может быть полезной новая строка форматирования sprintf %i — она выводит описание, соответствующее коду ошибки. Например,

```
r = -E_NO_MEM;
panic("env_alloc: %i", r);
```

остановит работу ядра с сообщением «env_alloc: out of memory».

Ниже приведен граф вызовов кода до точки, в которой запускается код процесса. Убедитесь, что вы понимаете цели каждого шага.

- start (kern/entry.S)
- i386_init (kern/init.c)
- cons_init
- fb_init()
- env_init
- env_create
- sched_yield
 - env_run
 - env_pop_tf

После реализации всех вышеупомянутых функций следует скомпилировать и запустить ядро. Система должна запустить двоичное приложение `test1`, которое проработает до первой инструкции `push`. Так как стек процесса не был задан (это будет сделано в следующем задании), будет сгенерировано исключение общей защиты. Ядро обнаружит, что его оно также не может обработать, что создает двойное исключение, которое на данном этапе также не обрабатывается. Наконец, процессор генерирует «тройную ошибку» (triple fault). Обычно в таких случаях процессор сбрасывается и система перезагружается.

В ближайшее время эта проблема будет решена, но сейчас можно использовать отладчик, чтобы убедиться, что осуществляется передача управления процессу. Запустите `JOSLLVM=0` (или `JOSLLVM=1`) `make qemu-gdb` и установите точку прерывания в `env_por_tf`, которая передает управление процессу. Пройдите эту функцию с помощью команды `si`; непосредственно передачу управления процессу производит инструкция `ret`. Система должна сгенерировать тройную ошибку раньше нее, на инструкции `pushq` — она записывает некоторые данные в стек, но адрес стека в этот момент еще не задан. Если ошибка происходит раньше, код управления процессами или загрузки программы ошибочен.

Упражнение №2

В функции `env_alloc` (`kern/env.c`) содержатся строки:

```
// LAB 3: Your code here:
static uintptr_t stack_top = 0x2000000;
```

Закомментированная строка задает адрес стека для процесса. Раскомментируйте ее и исправьте таким образом, чтобы для каждого процесса (т.е. при каждом вызове `env_alloc`) задавался уникальный адрес стека.

Примечание: Не имеет смысла выделять под стек большие объемы памяти. Как правило, должно хватать двух страничных кадров.

После выполнения этого упражнения программы должны работать до первой попытки вызова любой функции ядра (`cprintf`, `sys_yield`, `sys_exit`). Эти функции в программах представляют собой глобальные указатели, которые еще не инициализированы и равны нулю. Это проблема решается в дальнейших упражнениях.

Планировщик

Для поддержки переключения процессов необходим модуль, который определяет очередность работы процессов на основании некоторого алгоритма. Такой модуль называется планировщиком. Существуют различные алгоритмы планирования процессов, каждый из которых выполняет свои задачи и имеет свои положительные и отрицательные стороны. Например, в ОС Linux используется планировщик CFS (Completely Fair Scheduler), основной задачей которого является максимизация использования процессорного времени при одновременном сохранении максимальной интерактивности системы. Одним из простейших алгоритмов планирования процессов является Round-robin: в нем процессы выбираются для выполнения циклически по очереди, без использования приоритетов и других эвристик. Именно этот алгоритм следует реализовать для планирования процессов в JOS.

Упражнение №3

В функции `sched_yield` (`kern/sched.c`) должен быть реализован алгоритм планировщика. В данном случае используется простейший алгоритм Round-robin. Необходимо чтобы учитывалось как состояние процессов `ENV_RUNNABLE`, так и `ENV_RUNNING`. Исходите из того, что в системе JOS не поддерживается процессорной многоядерности. Обратите внимание на комментарии в функции `sched_yield`.

Упражнение №4

В файле `kern/entry.S` присутствует функция `sys_yield`. Эту функцию процессы вызывают для добровольной передачи управления ядру. В ней происходит сохранение контекста процесса, переключение на контекст ядра и передача управления планировщику. В этом же файле присутствует функция `sys_exit`, вызываемая процессом для завершения работы.

Допишите `sys_exit` по аналогии с `sys_yield` (примечание: в `sys_exit` сохранение контекста процесса, отдающего управление, не требуется).

Связывание

Упражнение 5.

В функции `load_icode` (`kern/env.c`) помимо загрузки образа процесса по необходимым ему адресам (эта задача была решена в предыдущих упражнениях) необходимо реализовать связывание (функция `bind_functions`) для загружаемых программ, чтобы они могли вызывать функции ядра.

Ваша задача состоит в реализации кода, который по адресам глобальных указателей на функции запишет адреса функций ядра, чтобы процессам был доступен любой запрошенный набор функций. Для этого необходимо сначала получить адреса глобальных переменных с конкретными названиями. Сейчас они получены из файлов `obj/prog/test1.sym` и `obj/prog/test2.sym`, и могут не совпадать с теми, которые будут сгенерированы в вашем случае. В функции `bind_functions` нужно получать адреса глобальных переменных (подобно тому, как это делают программы `nm -n`, `objdump -x`), и каждой глобальной переменной, чье имя совпадает с именем одной из функций ядра, присваивать адрес последней. Адреса функций ядра должны быть получены из отладочной информации, с которой вы уже научились работать в прошлой лабораторной работе. Для этого необходимо реализовать функции `find_function` и `dwarf_find_function` (`kern/kdebug.c`) и дополнить функцию `address_by_fname` (`kern/dwarf.c`) нахождением значения атрибута `DW_AT_low_pc`, содержащего адрес функции.

Примечание: обратите внимание на функции в файле `lib/string.c`, на раздел Symbol Table в спецификации Elf формата; строки, не завершенные нулем, можно легко вывести с помощью функции `printf` следующим образом: вызов `printf("%s", length, string)`; выведет максимум `length` байт из строки `string` (более подробную информацию о работе функции `printf` можно получить на её [man-странице](#)).

После реализации всех заданий, при выполнении `JOSLLVM=0` (или `JOSLLVM=1`) `make qemu` на экран должна выводиться примерно следующая информация:

```
[00000000] new env 00001000
[00000000] new env 00001001
[00000000] new env 00001002
[00001000] env started: RUNNABLE
[00001000] env stopped: RUNNING
[00001001] env started: RUNNABLE
TEST2 LOADED.
[00001001] env stopped: RUNNING
[00001002] env started: RUNNABLE
[00001002] env stopped: RUNNING
[00001000] env started: RUNNABLE
[00001000] env stopped: RUNNING
[00001001] env started: RUNNABLE
[00001001] env stopped: RUNNING
[00001002] env started: RUNNABLE
```

```
[00001002] env stopped: RUNNING
[00001000] env started: RUNNABLE
[00001000] env stopped: RUNNING
[00001001] env started: RUNNABLE
[00001001] env stopped: RUNNING
[00001002] env started: RUNNABLE
[00001002] env stopped: RUNNING
[00001000] env started: RUNNABLE
[00001000] free env 00001000
[00001000] env stopped: FREE
[00001001] env started: RUNNABLE
[00001001] env stopped: RUNNING
[00001002] env started: RUNNABLE
[00001002] free env 00001002
[00001002] env stopped: FREE
[00001001] env started: RUNNABLE
[00001001] env stopped: RUNNING
[00001001] env started: RUNNING
[00001001] free env 00001001
Halt
No runnable environments in the system!
Welcome to the JOS kernel monitor!
Type 'help' for a list of commands.
K>
```

По окончании выполнения работы следует сохранить внесенные изменения и отправить их в удаленный репозиторий.

Updated by [Vitaly Cheptsov](#) 5 months ago · 1 revisions